FFT – Fast Fourier Transform

Numerical Analysis E3, I3, FMN050
Numerical Analysis
Centre for Mathematical Sciences
Lund University, Sweden

URL: `http://www.maths.lth.se/na/`

May 15, 2006

Fourier series

A periodic function can be expanded in a Fourier series

$$f(t) = \sum_{k=0}^{\infty} c_k \mathrm{e}^{2\pi \mathrm{i} k t/T}$$

L^2-approximation on $[0, T]$:

Expansion	$f(t) = \sum_0^\infty c_k \varphi_k(t)$
Inner product	$\langle u, v \rangle = \frac{1}{T} \int_0^T \bar{u} v \, \mathrm{d}t$
Basis functions	$\varphi_k(t) = \mathrm{e}^{2\pi \mathrm{i} k t/T}$
Orthonormal basis	$\langle \varphi_k, \varphi_m \rangle = \delta_{km}$
Fourier coefficients	$c_m = \langle \varphi_m, f \rangle$

$$c_m = \frac{1}{T} \int_0^T \mathrm{e}^{-2\pi \mathrm{i} m t/T} f(t) \, \mathrm{d}t$$

Fourier series . . .

Fourier series expansion implies that the function f on $[0, T]$ is represented by an infinite sequence $\{c_k\}_0^\infty$.

For simplicity, put $T := 1$. How would one compute

$$c_m = \int_0^1 \mathrm{e}^{-2\pi \mathrm{i} m t} f(t)\,\mathrm{d}t\,; \qquad m = 0, 1, \ldots, \infty$$

Numerical integration (trapezoidal rule):

Introduce equidistant grid $t_n = n \cdot \Delta t \equiv n/N \quad \Rightarrow$

$$c_m \approx \frac{1}{N}\left[\frac{f(0)}{2} + \sum_{n=1}^{N-1} \mathrm{e}^{-2\pi \mathrm{i} m n/N} f(n/N) + \frac{f(1)}{2}\right]$$

But f periodic implies $f(0) = f(1)$, so

$$c_m \approx \frac{1}{N} \sum_{n=0}^{N-1} \mathrm{e}^{-2\pi \mathrm{i} m n/N} f(n/N)\,; \qquad m = 0, \ldots, N-1$$

Discrete Fourier Transform

Consider

$$c_m \approx \frac{1}{N} \sum_{n=0}^{N-1} \mathrm{e}^{-2\pi \mathrm{i} mn/N} f(n/N)$$

Sequence of function values: $\mathbf{f} = \{f(n/N)\}_{n=0}^{N-1}$

Fourier coefficient vector: $\mathbf{c} = \{c_m\}_{m=0}^{N-1}$

Introduce the $N \times N$ **matrix**

$$\mathcal{F}_N = \left\{ \frac{\mathrm{e}^{-2\pi \mathrm{i} mn/N}}{N} \right\}_{m,n=0}^{N-1}$$

Then the matrix–vector multiplication

$$\mathbf{c} = \mathcal{F}_N \cdot \mathbf{f}$$

is the **Discrete Fourier Transform (DFT)**.

Discrete Fourier Transform . . .

Let $N \to \infty$ in

$$\mathbf{c} = \mathcal{F}_N \cdot \mathbf{f}$$

to get more Fourier coefficients, at increasing accuracy

But the DFT is interesting in its own right!

If f is only known on discrete points one can

- **only** compute a DFT
- **only** compute a finite number of frequencies

Why? Sampling theorem; Nyquist frequency.

If f is only known on discrete points the DFT

- is no longer an approximation,
- but is the exact information available

Computation of DFT

Simplest case: $N = 2$; $\quad \mathcal{F}_2 = \{\mathrm{e}^{-2\pi \mathrm{i} mn/2}/2\}$

$$\mathbf{c} = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \mathbf{f}$$

Note extremely simple structure:

$$\begin{aligned} c_0 &= (f_0 + f_1)/2 \\ c_1 &= (f_0 - f_1)/2 \end{aligned}$$

This is called a **butterfly**

Many computers have this operation in hardware

(addition and subtraction + shift)

Computation of DFT . . .

The case $N = 4$; $\quad \mathcal{F}_4 = \{\mathrm{e}^{-2\pi \mathrm{i} mn/4}/4\}$

$$\mathbf{c} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -\mathrm{i} & -1 & \mathrm{i} \\ 1 & -1 & 1 & -1 \\ 1 & \mathrm{i} & -1 & -\mathrm{i} \end{pmatrix} \cdot \mathbf{f}$$

Splitting and permutation

$$\mathbf{f}_{\text{even}} = \begin{pmatrix} f_0 \\ f_2 \end{pmatrix} \qquad \mathbf{f}_{\text{odd}} = \begin{pmatrix} f_1 \\ f_3 \end{pmatrix}$$

$$\mathbf{c}_{\text{low}} = \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} \qquad \mathbf{c}_{\text{high}} = \begin{pmatrix} c_2 \\ c_3 \end{pmatrix}$$

Then

$$\begin{pmatrix} \mathbf{c}_{\text{low}} \\ \mathbf{c}_{\text{high}} \end{pmatrix} = \frac{1}{2} \begin{pmatrix} \mathcal{F}_2\, \mathbf{f}_{\text{even}} + \Phi\, \mathcal{F}_2\, \mathbf{f}_{\text{odd}} \\ \mathcal{F}_2\, \mathbf{f}_{\text{even}} - \Phi\, \mathcal{F}_2\, \mathbf{f}_{\text{odd}} \end{pmatrix}$$

with phase factor $\Phi = \begin{pmatrix} 1 & 0 \\ 0 & -\mathrm{i} \end{pmatrix}$, always diagonal

The fast Fourier transform FFT

$\mathcal{F}_2 =$ one butterfly

$\mathcal{F}_4 =$ linear combination of 2 butterflies

$\mathcal{F}_8 =$ linear combination of two $\mathcal{F}_4$, or 4 butterflies

...

$\mathcal{F}_{2^p} =$ linear combination of 2^p butterflies

One butterfly is done in 2 flops

Total work for FFT with $N = 2^p$ data is

$$N \log_2 N = pN \quad \text{flops}$$

Note: Computing $\mathcal{F}_N \mathbf{f}$ as a standard matrix–vector multiply costs N^2 operations!

Example: $N = 2^{20}$ (1 Meg data)

Matrix–vector multiply costs approx 10^{12} flops (10 h)

FFT costs only $20 \cdot 2^{20} \approx 2 \cdot 10^7$ flops (< 1 sec)

Trigonometric polynomials

Fourier series of periodic function on $[0, 1]$

$$f(t) = \sum_{k=0}^{\infty} c_k \mathrm{e}^{2\pi \mathrm{i} k t}$$

Approximation using finite number of terms ($c_k \to 0$):

$$f(t) \approx \sum_{k=0}^{N-1} c_k \mathrm{e}^{2\pi \mathrm{i} k t}$$

Put $z = \mathrm{e}^{2\pi \mathrm{i} t} = \cos 2\pi t + \mathrm{i} \sin 2\pi t$. Then

$$P_{N-1}(z) := \sum_{k=0}^{N-1} c_k z^k$$

is a *trigonometric polynomial* of degree $N - 1$, and

$$f(t) \approx P_{N-1}(z)$$

Trigonometric interpolation

How determine coefficients c_k so that $f(t) \approx P_{N-1}(z)$?

Trigonometric interpolation: Require

$$f(t_n) = P_{N-1}(z_n)$$

for t_n with $z_n = \mathrm{e}^{2\pi \mathrm{i} t_n}$ (where $n = 0, \ldots, N-1$)

Equidistant trigonometric interpolation: $t_n := n/N$
Solve the N equations

$$f(n/N) = \sum_{k=0}^{N-1} c_k z_n^k$$

for the N coefficients $\mathbf{c} = \{c_k\}_0^{N-1}$:

$$\mathbf{f} = Z\mathbf{c} \quad \Rightarrow \quad \mathbf{c} = Z^{-1}\mathbf{f}$$

with matrix elements $Z_{nk} = \{z_n^k\}$. But $Z^{-1} = \mathcal{F}_N$!

DFT and its inverse

Theorem: *The DFT matrix*

$$\mathcal{F}_N = \left\{ \frac{\mathrm{e}^{-2\pi \mathrm{i} mn/N}}{N} \right\}_{m,n=0}^{N-1}$$

has the inverse

$$\mathcal{F}_N^{-1} = \left\{ \mathrm{e}^{2\pi \mathrm{i} mn/N} \right\}_{m,n=0}^{N-1}$$

Notes:

- $\mathcal{F}_N$ is symmetric
- Both FFT and inverse FFT run in $N \log_2 N$ flops
- The normalizing factor N can be put either in $\mathcal{F}_N$ or its inverse (both versions of FFT occur)
- If one puts $\sqrt{N}$ in both, one gets a *unitary* DFT

FFT as a unitary transformation

Balanced DFT matrix

$$\hat{\mathcal{F}}_N = \left\{ \frac{\mathrm{e}^{-2\pi \mathrm{i} mn/N}}{\sqrt{N}} \right\}_{m,n=0}^{N-1}$$

has the inverse

$$\hat{\mathcal{F}}_N^{-1} = \left\{ \frac{\mathrm{e}^{2\pi \mathrm{i} mn/N}}{\sqrt{N}} \right\}_{m,n=0}^{N-1}$$

This implies that $\hat{\mathcal{F}}_N$ is *unitary*:

$$\hat{\mathcal{F}}_N^{-1} = \hat{\mathcal{F}}_N^{\mathrm{H}} \quad \Rightarrow \quad \|\hat{\mathcal{F}}_N\|_2 = \|\hat{\mathcal{F}}_N^{-1}\|_2 = 1$$

Therefore the condition number $\kappa_2[\hat{\mathcal{F}}_N] \equiv 1$

FFT is always perfectly well conditioned!

FFT as a unitary transformation

Unitary transformations do not change the 2-norm:

$$y = Ax, \quad A^{-1} = A^{\mathrm{H}} \;\Rightarrow$$

$$\|y\|_2^2 = y^{\mathrm{H}} y = x^{\mathrm{H}} A^{\mathrm{H}} A x = x^{\mathrm{H}} x = \|x\|_2^2$$

Theorem (Parseval):

$$\hat{\mathbf{c}} = \hat{\mathcal{F}}_N \mathbf{f} \quad \Rightarrow \quad \|\hat{\mathbf{c}}\|_2 = \|\mathbf{f}\|_2$$

This is really just the Pythagorean theorem in disguise:

$$\sum_{0}^{N-1} \hat{c}_k^2 \;=\; \|\mathbf{f}\|_2^2$$